

Atividade — Álbum de Figurinhas (Full Stack)

Seu desafio é construir uma aplicação para gerenciamento de um **álbum de figurinhas digital**. O tema do álbum é livre — pode ser Copa do Mundo, Pokémon, super-heróis, animais, filmes, bandas, personagens históricos, ou qualquer outro tema de sua preferência. O importante é que a aplicação demonstre as funcionalidades descritas a seguir.

A aplicação deve ser composta por um **REST API em Node.js + Express** no backend e uma **aplicação web em React + TanStack** no frontend.

Backend — REST API (Node.js + Express)

Domínio da aplicação

A aplicação trabalha com **duas ideias distintas** que precisam ser representadas no domínio:

1. Catálogo de figurinhas

Representa o conjunto de figurinhas que existem no álbum, independentemente de qual colecionador as possui. Ele define o que pode ser colecionado.

Cada figurinha do catálogo precisa cadastrar pelo menos:

- **número** — identificador da figurinha dentro do álbum (ex: 1, 2, 3...). É único no catálogo, não é possível cadastrar duas figurinhas com o mesmo número
- **nome** — nome da figurinha (jogador, personagem, item, etc., de acordo com o tema escolhido)
- **categoria** — uma classificação dentro do tema (ex: seleção, time, tipo, região, etc.). Defina um conjunto fechado de categorias possíveis e documente quais são
- **raridade** — comum, rara ou lendária. Caso não seja informada, preencher com comum
- **URL da imagem** — link público para a imagem da figurinha. Para esta versão inicial, pode ser uma URL de uma imagem hospedada na internet (sem necessidade de upload próprio). Boas fontes de imagens públicas incluem **Wikimedia Commons**, **Unsplash**, **Pexels**, **Pixabay** ou imagens do **Wikipedia** correspondentes ao tema. O [desafio opcional de upload](#) tratará da hospedagem em storage próprio.

Outros campos opcionais (descrição, ano, etc.) podem ser adicionados conforme o tema escolhido.

2. Álbum do colecionador

Representa **quais figurinhas do catálogo um colecionador específico possui e em que quantidade**. Para esta versão inicial, considere um único colecionador padrão do sistema (o [desafio de autenticação](#) tratará da separação por usuário).

O álbum precisa capturar pelo menos:

- a relação entre o colecionador e cada figurinha do catálogo
- informação suficiente para distinguir, para cada figurinha do catálogo, se o colecionador **não a possui** (faltando), se já a **possui** (colada) ou se possui **cópias extras** (disponíveis para troca)

Nota: A escolha do banco é livre (PostgreSQL, MySQL, MongoDB, SQLite, etc.).

API

A API deve expor as operações necessárias para que o frontend cumpra todos os requisitos descritos adiante. No mínimo, deve ser possível:

Sobre o catálogo:

- criar uma nova figurinha no catálogo
- consultar figurinhas do catálogo, com suporte a filtros combináveis por número, categoria e/ou raridade
- atualizar os dados de uma figurinha do catálogo
- remover uma figurinha do catálogo — defina e documente o comportamento esperado quando a figurinha estiver presente no álbum de algum colecionador (bloquear remoção, remover em cascata, etc.)

Sobre o álbum do colecionador:

- registrar que o colecionador obteve uma figurinha do catálogo (incluindo o caso de obter cópias adicionais que se tornam repetidas)
- remover uma figurinha do álbum
- consultar as figurinhas do álbum, com suporte a filtros combináveis por categoria, raridade e/ou status de coleção (faltando, colada, repetida)
- consultar as estatísticas de progresso do álbum em uma única chamada (total no catálogo, total colado, total faltando, total de repetidas para troca, percentual de progresso)

Importante: o **desenho dos endpoints** (rotas, métodos HTTP, formato dos payloads, códigos de status, estrutura das respostas) **faz parte do desafio**.

Frontend — React + TanStack

A interface do colecionador deve ser construída utilizando **React** com a stack TanStack:

- **TanStack Query** para gerenciamento de cache e sincronização com o backend
- **TanStack Router** para navegação entre as telas
- **TanStack Table** para listagem e manipulação das figurinhas

Telas obrigatórias

1. Dashboard / Página inicial Exibe um resumo da coleção com as estatísticas retornadas pelo endpoint de estatísticas: progresso geral (barra de progresso), número de figurinhas coladas, número de figurinhas faltando e número de repetidas disponíveis para troca.

2. Álbum Visualização do álbum do colecionador no formato de **páginas de álbum físico**, com as figurinhas agrupadas por categoria (cada categoria forma uma "página" ou seção do álbum) e ordenadas pelo número dentro de cada categoria. Cada figurinha deve indicar visualmente seu status:

- **faltando** — placeholder/silhueta indicando o espaço vazio no álbum
- **colada** — figurinha destacada (preenchida)
- **repetida** — figurinha colada com badge indicando a quantidade extra (ex: +2)

Esta tela é a visão principal do progresso do colecionador e deve refletir a estrutura do álbum: agrupada, ordenada e visualmente clara sobre o que falta e o que já foi obtido.

3. Listagem de figurinhas Tabela com todas as figurinhas cadastradas usando TanStack Table, com:

- ordenação por número, nome e categoria
- filtros por categoria, raridade e status (faltando / colada / repetida)
- paginação client-side ou server-side (a escolha do candidato, justificando no README)
- ações inline para incrementar/decrementar a quantidade de figurinhas (botões + e -), editar e remover

4. Cadastro / Edição de figurinha Formulário para criar uma nova figurinha ou editar uma existente. Todos os campos devem ser validados no frontend antes do envio, com mensagens de erro claras.

5. Detalhes da figurinha Tela individual mostrando todos os dados da figurinha, incluindo opções para incrementar/decrementar a quantidade e indicador visual do status (faltando, colada, repetida).

Requisitos de UX

- Estados de **loading** durante chamadas à API
- Estados de **erro** com mensagens amigáveis ao usuário
- Estados de **vazio** (ex: "nenhuma figurinha cadastrada ainda")
- Mutations otimistas onde fizer sentido (incrementar/decrementar quantidade é um forte candidato), com rollback em caso de erro
- Cache invalidation correta após mutations — após criar, editar ou deletar uma figurinha, a lista e as estatísticas devem ser atualizadas

Desafios Opcionais

Desafio Goleiro: Defenda sua API

O desafio consiste na melhoria da robustez da API. Todas as entradas devem ser validadas para garantir que os dados passados pelo payload sejam do formato esperado:

- Campos obrigatórios faltantes devem ser notificados
- Campos extras devem ser desconsiderados
- Campos com tipagens erradas devem ser notificados (string no lugar de número, ou um valor diferente dos possíveis enums)
- Quantidade negativa deve ser rejeitada

Recomenda-se o uso de uma biblioteca de validação de schema (Zod, Joi, Yup ou similar). Lembre-se de seguir os padrões REST referentes a Status Code.

Desafio Cofre: Cada um cuida do seu álbum

O desafio consiste em implementar um **sistema básico de autenticação de usuários** e tornar o álbum **gerenciado por usuário**. Cada colecionador deve ter seu próprio álbum, isolado dos demais — mas o catálogo de figurinhas continua sendo compartilhado entre todos.

Requisitos:

- Endpoints de **cadastro** e **login** com e-mail e senha
- Senhas armazenadas com hash (bcrypt, argon2 ou similar) — nunca em texto puro
- Autenticação baseada em **JWT** ou sessão, à escolha do candidato (justifique no README)
- Endpoints do **álbum** passam a exigir autenticação
- Cada entrada do álbum pertence a um usuário; um usuário só pode ver, editar ou deletar as entradas do **seu próprio álbum**
- O **catálogo de figurinhas** continua acessível para leitura por qualquer usuário autenticado. Quanto à escrita no catálogo (criar/editar/deletar figurinhas), defina e justifique a política: pode ser aberta para qualquer usuário, restrita a um perfil de administrador, ou bloqueada para todos
- No frontend, criar telas de **login** e **cadastro**, rotas protegidas via TanStack Router e um botão de **logout**
- O token/sessão deve ser persistido entre reloads da página

Desafio Coleção Visual: Hospede suas próprias figurinhas

Por padrão, o catálogo guarda apenas a URL de imagens públicas hospedadas em terceiros — o que é frágil (links podem quebrar, imagens podem ser removidas). O desafio consiste em **permitir o upload das imagens** diretamente para um sistema de **storage de objetos** controlado pela aplicação, substituindo as URLs externas por URLs servidas pelo próprio sistema.

Requisitos:

- Endpoint de upload aceitando **multipart/form-data**
- Validação de **tipo** (apenas imagens — png, jpg, webp) e **tamanho máximo** (sugestão: 5MB)
- Armazenamento em um sistema de object storage — pode ser **AWS S3**, **Google Cloud Storage**, **Cloudflare R2**, ou um equivalente local como **MinIO** rodando via Docker (recomendado para evitar custos durante o desenvolvimento). Não salvar a imagem no sistema de arquivos local do servidor nem no banco de dados como blob
- O campo de URL da imagem no banco passa a apontar para o objeto no storage próprio
- Ao deletar uma figurinha, o objeto correspondente deve ser removido do storage
- Adaptar o formulário de figurinhas do frontend para lidar com o upload de imagens no novo endpoint

Desafio Trocador: A magia do álbum está nas trocas

Implemente um endpoint que sugira trocas entre dois colecionadores. Dado um colecionador A e um colecionador B, a API deve propor uma troca balanceada de figurinhas entre ambos, considerando a raridade das figurinhas sugeridas.

No frontend, crie uma tela dedicada que mostra as trocas sugeridas em formato de cards lado a lado.

Este desafio se beneficia muito do Desafio Cofre estar implementado, já que pressupõe múltiplos usuários.

Desafio Estádio: Coleção bonita rende mais figurinha

O desafio consiste em melhorar o frontend com:

- **Polimento da tela de Álbum:** ao invés de placeholders genéricos para figurinhas faltantes, criar silhuetas estilizadas; adicionar separadores visuais entre as "páginas" (categorias) com cabeçalhos temáticos; efeito de "virar página" ou navegação por categoria; visual que remeta a um álbum físico (papel, sombras, espaçamentos)
- **Dark mode** com toggle persistido (localStorage)
- Animações sutis ao incrementar uma figurinha (a primeira "colagem" merece um destaque visual)

 **Desafio Cebolinha:** O plano tem que ser infalível

O desafio consiste na implementação de casos de teste para validar o comportamento da aplicação. Backend pode ser testado com Jest ou similar; frontend com Vitest, Cypress ou similar.

Testes recomendados incluem:

Backend:

- Cadastro de figurinha sendo realizado com sucesso
- Cadastro falhando em caso de número duplicado
- Cadastro falhando em caso de categoria/raridade inválida
- Recuperação de figurinha existente e tratamento de figurinha inexistente
- Filtros funcionando individualmente e combinados
- Atualização e remoção sendo realizadas com sucesso
- Endpoint de estatísticas retornando valores corretos
- (Se Cofre implementado) Bloqueio de acesso para usuário não autenticado e isolamento entre usuários

Frontend:

- Renderização da listagem com dados mockados
- Submissão do formulário de cadastro com dados válidos
- Exibição de mensagens de erro com dados inválidos
- Invalidação de cache após mutation

 **Desafio Hat-trick:** Entrega impecável

Disponibilize a aplicação rodando — backend, frontend, banco de dados e (se aplicável) storage — através de **Docker Compose**, com um único `docker-compose up` levantando todo o ambiente. Inclua um README com instruções claras de execução, decisões de arquitetura e justificativas técnicas.

Dados iniciais

A aplicação deve iniciar com dados pré-cadastrados (seed) seguindo o tema escolhido:

Catálogo: pelo menos **10 figurinhas** cadastradas no catálogo, contemplando diferentes categorias e raridades (incluindo pelo menos uma lendária).

Álbum do colecionador padrão: pelo menos **5 entradas** no álbum, contemplando diferentes status — algumas figurinhas do catálogo ainda faltando, algumas coladas e pelo menos uma com repetida.

Se o Desafio Cofre estiver implementado, criar pelo menos **2 usuários de exemplo** com álbuns distintos sobre o **mesmo catálogo**, para facilitar a demonstração e abrir caminho para o Desafio Trocador.

Critérios de avaliação

- Aderência aos requisitos funcionais (backend e frontend)
 - **Modelagem de dados** — como o domínio foi traduzido em entidades, relações e estrutura no banco
 - **Aderência às boas práticas REST** — desenho dos endpoints, uso semântico de métodos HTTP, status codes apropriados, nomeação de recursos
 - Organização do código e separação de responsabilidades
 - Uso adequado das ferramentas TanStack no frontend
 - Tratamento de erros e estados (loading, erro, vazio)
 - **Responsividade** — a aplicação deve funcionar bem em celular, tablet e desktop
 - **Boas práticas de UX** — hierarquia visual clara, feedback adequado às ações do usuário, acessibilidade básica, consistência visual
 - Qualidade do README e documentação das decisões técnicas
 - Implementação dos desafios opcionais (diferencial, não obrigatório)
-

Entrega

Prazo: a atividade deve ser entregue até o dia **12/06**.

Formato: a entrega deve ser feita por meio de um **repositório público no GitHub** contendo o código do **backend e do frontend no mesmo repositório** (sugestão de estrutura: pastas **backend/** e **frontend/** na raiz, ou nomes equivalentes que deixem clara a separação).

README: o repositório precisa conter um arquivo **README.md** na raiz com pelo menos:

- descrição breve do projeto e do tema escolhido para o álbum
- instruções claras de **como rodar a aplicação localmente** (backend, frontend e banco de dados), incluindo pré-requisitos, variáveis de ambiente e comandos
- **decisões técnicas** e justificativas — escolha do banco de dados, biblioteca de validação, estratégia de autenticação (se aplicável), modelagem do domínio, paginação client-side vs. server-side, política de remoção do catálogo, etc.
- lista de quais **desafios opcionais** foram implementados
- limitações conhecidas, pontos em aberto ou trade-offs relevantes